CS 411 Stage 2: Conceptual and Logical Database Design

Project: Virtually Integrated Agenda (VIA)

Team 001 - TableForFour
Max Bromberg (mbrom3)
Zainab Memon (mzainab2)
Meenakshi De (mde6)
Jonathan Martinez-Herrada (jmart466)

February 27th, 2026

1 Entity Relationship Diagram

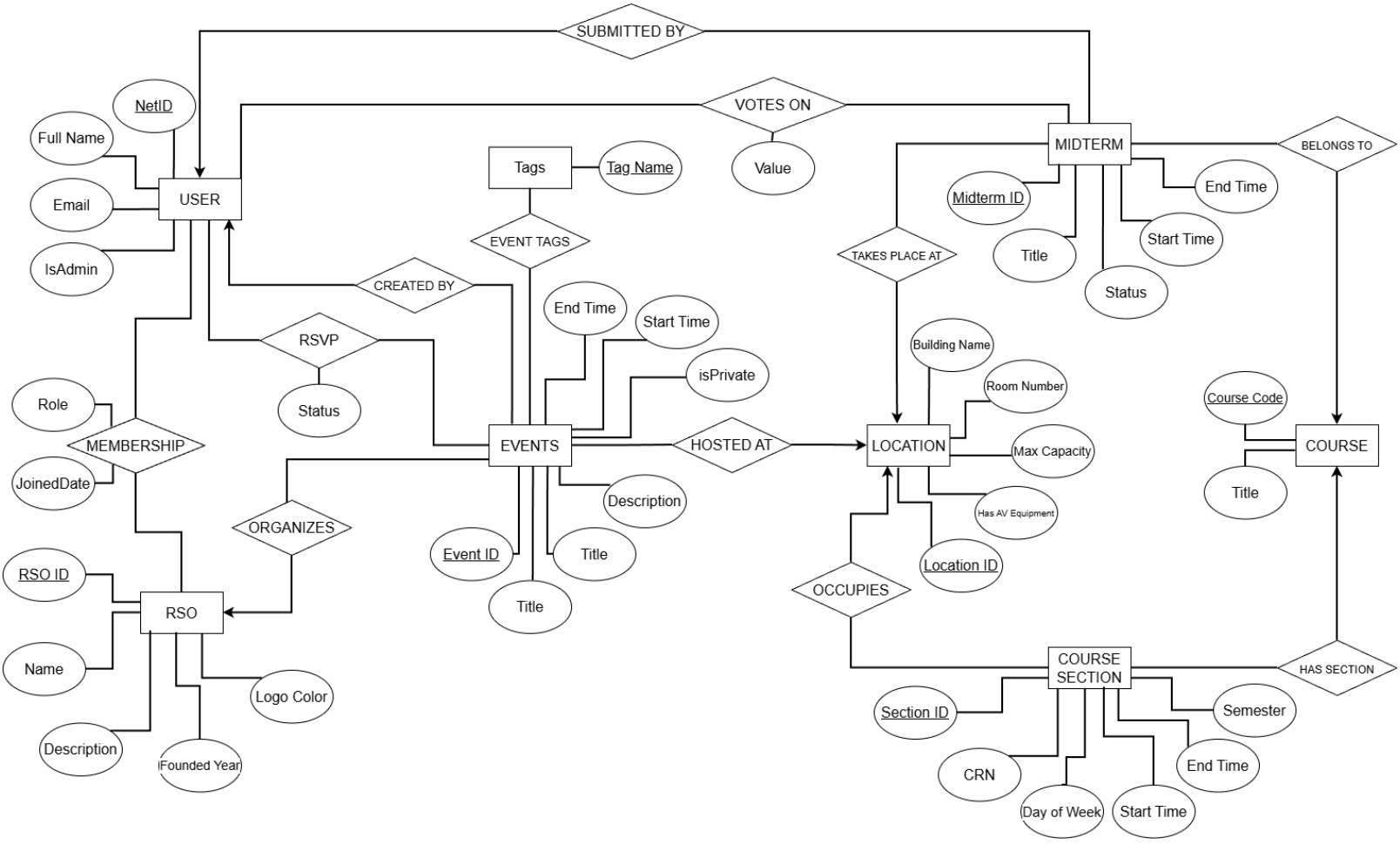

Figure 1: Entity Relationship Diagram

2 Our Assumptions: Entity vs. Attribute Design Decisions

This section explains the assumptions made in our ER model and justifies why certain components were modeled as entities instead of attributes. Our decisions were guided by normalization principles, relationship complexity, and system functionality (especially scheduling and conflict detection).

2.1 Entities:

2.1.1 User

- Assumptions:
    - Each user has a unique NetID.
    - A user may belong to multiple RSOs.
    - A user may RSVP to multiple events.
    - A user may submit and vote on midterms.
    - Users may have administrative privileges.
- Why modeled as an entity:
    - User represents an independent system actor with multiple relationships across the schema. Because users interact with RSOs, Events, Midterms, and Courses, and have their own attributes and permissions, User must exist as its own entity rather than as an attribute of another entity.

2.1.2 RSO

- Assumptions:
    - An RSO has a unique RSO_ID.
    - An RSO organizes events.
    - An RSO has many members.
    - Membership contains attributes such as Role and JoinedDate.
- Why modeled as an entity:
    - Although we initially considered making RSO an attribute of User, this would not support many-to-many membership or membership-specific attributes. Since RSOs have their own attributes and organize events, they must exist independently as entities.

2.1.3 Event

- Assumptions:
    - Each event has a unique Event_ID.
    - An event is organized by one RSO.
    - An event occurs at one Location.
    - An event may have multiple Tags and RSVPs.
    - An event may or may not have AV equipment
    - An event may or may not be Private (open to only RSO members or Board)
- Why modeled as an entity:
    - Events contain multiple attributes (time, description, location) and participate in several relationships. They represent scheduled objects in the system and therefore cannot be attributes of RSO or Location.

2.1.4 Location

- Assumptions:
  - Each physical room has a unique Location_ID.
  - A location has capacity and identifying details.
  - Locations are shared by Events, Midterms, and Course Sections.
- Why modeled as an entity:
  - If Location were stored as a string attribute (e.g., "ECEB 1002"), conflict detection would require unreliable string matching. Because multiple entities reference the same physical space and scheduling conflicts must be prevented, Location must be modeled as its own entity to ensure referential integrity.

2.1.5 Midterm

- Assumptions:
  - A midterm belongs to one course.
  - A midterm occurs at one location.
  - A midterm can receive votes.
  - A midterm can be submitted by users.
- Why modeled as an entity:
  - Midterms have independent attributes (time, status, location) and participate in multiple relationships (votes, submissions, course association). Therefore, they require their own identity.

2.1.6 Tag

- Assumptions:
  - An event may have multiple tags.
  - A tag may apply to multiple events.
- Why modeled as an entity:
  - Storing tags as comma-separated strings would violate First Normal Form and make filtering inefficient. Making Tag an entity enforces consistency and supports a many-to-many relationship.

2.1.7 Course

- Assumptions:
  - A course is an abstract academic concept.
  - A course has multiple sections.
  - A course may have multiple midterms.
- Why modeled as an entity:
  - Courses represent academic offerings independent of time and location. Storing course data within sections would cause redundancy and violate normalization principles.

2.1.8 Course Section

- Assumptions:
  - A section represents a physical instance of a course.

    - A section has a time, location, and CRN.
    - A section belongs to exactly one course.
- Why modeled as a separate entity from Course:
    - Sections have unique time and location attributes, while courses represent abstract content. Separating them avoids redundancy and supports scheduling logic.

2.2 Relationship Assumptions and Cardinalities

Below is the justification for the cardinality of each relationship, driven by the constraints of the "Via" application.

2.2.1 Community Relationships

- Membership (User ↔ RSO)
    - Cardinality: Many-to-Many (M:N)
    - Assumption:
        - A student can be involved in multiple organizations (e.g., a member of IEEE and President of HKN). An RSO inherently has many users. Membership is not binary — it includes attributes such as Role (Member, Board, Admin) and JoinedDate, which dictate permissions and tenure.
        - Because membership has attributes and both sides can have multiple instances, this must be modeled as a many-to-many relationship (implemented via an associative entity).
- Organizes (RSO ← Event)
    - Cardinality: One-to-Many (1:N)
        - (One RSO organizes many Events; each Event has exactly one organizing RSO.)
    - Assumption:
        - While co-hosting exists in reality, to simplify ownership and editing rights on the platform, we assume each event has exactly one primary organizing RSO. However, an RSO can host many events over time.
- Created By (User → Event)
    - Cardinality: One-to-Many (1:N)
        - (One User may create many Events; each Event is created by exactly one User.)
    - Assumption:
        - This relationship represents the individual account that physically creates the event entry in the system. While an Event is organized by exactly one RSO, the platform requires tracking which specific user initiated the creation for accountability, auditing, and moderation purposes.
        - A user may create multiple events (e.g., an RSO board member creating weekly meetings), but each event is associated with exactly one creator in the system.
- RSVP (User ↔ Event)
    - Cardinality: Many-to-Many (M:N)
    - Assumption:
        - A user may RSVP to multiple events, and an event may have many attendees. The relationship includes a Status attribute (Going, Maybe, Not

Going). Because this status is specific to each user-event pair, the relationship must be many-to-many.

- Categorized By (Event ↔ Tag)
    - Cardinality: Many-to-Many (M:N)
    - Assumption:
        - An event can have multiple tags (e.g., Free Food, Social, Corporate), and a tag can apply to many events. This supports filtering and avoids storing multi-valued attributes inside Event.

2.2.2 Scheduler Relationships (Infrastructure)

- Hosted At (Event → Location)
    - Cardinality: Many-to-One (N:1)
        - (Many Events may occur at one Location; each Event occurs at exactly one Location.)
    - Assumption:
        - An event happens in exactly one physical location at a given time. A location (e.g., ECEB 1000) hosts many events throughout the semester or year.
- Occupies (Course Section → Location)
    - Cardinality: Many-to-One (N:1)
    - Assumption:
        - A course section (lecture/lab/discussion) meets in exactly one room. A location hosts many sections across different time slots.
- Takes Place At (Midterm → Location)
    - Cardinality: Many-to-One (N:1)
    - Assumption:
        - A midterm is scheduled in exactly one location (including testing centers like CBTF, which we treat as a Location entity). A location may host many midterms across the semester.

2.2.3 Academic Relationships

- Belongs To (Midterm → Course)
    - Cardinality: Many-to-One (N:1)
        - (Many Midterms belong to one Course.)
    - Assumption:
        - A specific midterm (e.g., Exam 1) is tied to exactly one Course (e.g., ECE 313). A course may have multiple midterms over the semester.
- Has Section (Course ← Course Section)
    - Cardinality: One-to-Many (1:N)
        - (One Course has many Course Sections; each Section belongs to exactly one Course.)
    - Assumption:
        - A course is an abstract academic concept. Its sections (Lecture A, Lab B, etc.) are physically scheduled instances. Each section corresponds to exactly one course, but a course can have multiple sections.
- Submitted By (User ← Midterm)

    - Cardinality: One-to-Many (1:N)
        - (One User may submit many Midterms; each Midterm has exactly one submitting User.)
    - Assumption:
        - For accountability in the crowdsourcing feature, exactly one user is recorded as the submitter of each midterm entry. A helpful student may submit multiple midterms.
- Votes On (User ↔ Midterm)
    - Cardinality: Many-to-Many (M:N)
    - Assumption:
        - This is a critical application constraint. If "Upvotes" were simply an integer attribute of Midterm, users could vote infinitely. By modeling Votes On as a many-to-many relationship (with a Value attribute of +1 or −1), we enforce that a user can cast at most one vote per midterm. This preserves the integrity of the crowdsourced data.

3 ER Diagram Satisfies at least 5 entities (at most one representing user accounts) and at least two types of relationships

Our Entity Relationship Diagram and Relational Schema both reflect the completion of this requirement, with the current iteration of our database schema having more than the aforementioned requirement as well as an instance of a user account entity to reflect the students who would be using our application.

4 Schema adheres to 3NF

4.1 Adherence to 1NF

1NF requires that all attributes are atomic (indivisible), and there are no repeating groups or multi-valued attributes. The schema below meets requirements for 1NF because we explicitly avoided storing lists or arrays as strings. For example, instead of storing a comma-separated list of tags (i.e. "Free Food, Corporate") inside the Events table, we extracted this into the Event_Tags table where each tag is an atomic row. Similarly, a user's multiple RSO memberships are handled by distinct rows in the RSO_Memberhsips table rather than an array in the Users table.

4.2 Adherence to 2NF

2NF requires that the schema is 1NF, and that all non-prime attributes (attributes not part of the primary key) must be strictly and fully functionally dependent on the entire primary key (no partial dependencies). The schema below meets these requirements because partial dependencies only occur in tables with composite primary keys (Event_Tags, RSVPs, and Midterm_Votes). In RSVPs, the status relies on both the specific user and the specific event, it cannot depend on just the net_id or just the event_id. In Midterm_Votes, the vote_value depends entirely on the combination of the specific midterm and the specific user casting the vote. All single-key tables

(Users, Events) inherently satisfy 2NF because partial dependency is impossible when the key is a single attribute.

4.3 Adherence to 3NF

3NF requires that the schema is 2NF, and that there are no transitive dependencies. A non-prime attribute must not depend on another non-prime attribute (every non-key attribute must depend only on the primary key). The schema below meets this requirement by strictly separating concepts to prevent transitive chains. If we had included building, room_number, and max_capacity directly in the Events table, it would violate 3NF. max_capacity would depend on room_number, which in turn depends on the event_id. By normalizing Locations into its own table, Events only holds the location_id (foreign key). The capacity depends only on the location_id in the Locations table. Similarly, if the Midterms table included the course_title (e.g. “Computer Systems Engineering”), it would be transitively dependent. By referencing course_code as a foreign key, we ensure the title remains solely in the Courses table, completely satisfying 3NF.

5. Logical Database Design (Relational Schema)

```
Users(net_id:VARCHAR [PK], full_name:VARCHAR, email:VARCHAR,
is_global_admin:BOOLEAN)

RSOs(rso_id:INT [PK], name:VARCHAR, description:TEXT,
logo_color:VARCHAR, founded_year:INT)

RSO_Memberships(net_id:VARCHAR [PK] [FK to Users.net_id],
rso_id:INT [PK] [FK to RSOs.rso_id], role:VARCHAR,
joined_at:DATETIME)

Locations(location_id:INT [PK], building:VARCHAR,
room_number:VARCHAR, max_capacity:INT, has_av_equipment:BOOLEAN)

Courses(course_code:VARCHAR [PK], title:VARCHAR)

Course_Sections(section_id:INT [PK], course_code:VARCHAR [FK to
Courses.course_code], location_id:INT [FK to
Locations.location_id], day_of_week:VARCHAR, start_time:TIME,
end_time:TIME, semester:VARCHAR)

Events(event_id:INT [PK], rso_id:INT [FK to RSOs.rso_id],
created_by:VARCHAR [FK to Users.net_id], location_id:INT [FK to
Locations.location_id], title:VARCHAR, description:TEXT,
start_time:DATETIME, end_time:DATETIME, is_private:BOOLEAN)

Tags(tag_name:VARCHAR [PK])

Event_Tags(event_id:INT [PK] [FK to Events.event_id],
tag_name:VARCHAR [PK] [FK to Tags.tag_name])
```

RSVPs(net_id:VARCHAR [PK] [FK to Users.net_id], event_id:INT [PK] [FK to Events.event_id], status:VARCHAR)

Midterms(midterm_id:INT [PK], course_code:VARCHAR [FK to Courses.course_code], submitted_by:VARCHAR [FK to Users.net_id], location_id:INT [FK to Locations.location_id], title:VARCHAR, start_time:DATETIME, end_time:DATETIME, status:VARCHAR)

Midterm_Votes(midterm_id:INT [PK] [FK to Midterms.midterm_id], net_id:VARCHAR [PK] [FK to Users.net_id], vote_value:INT)